

Evaluación técnica:

Arquitectura propuesta

Estructura actual propuesta en el proyecto Water Tank

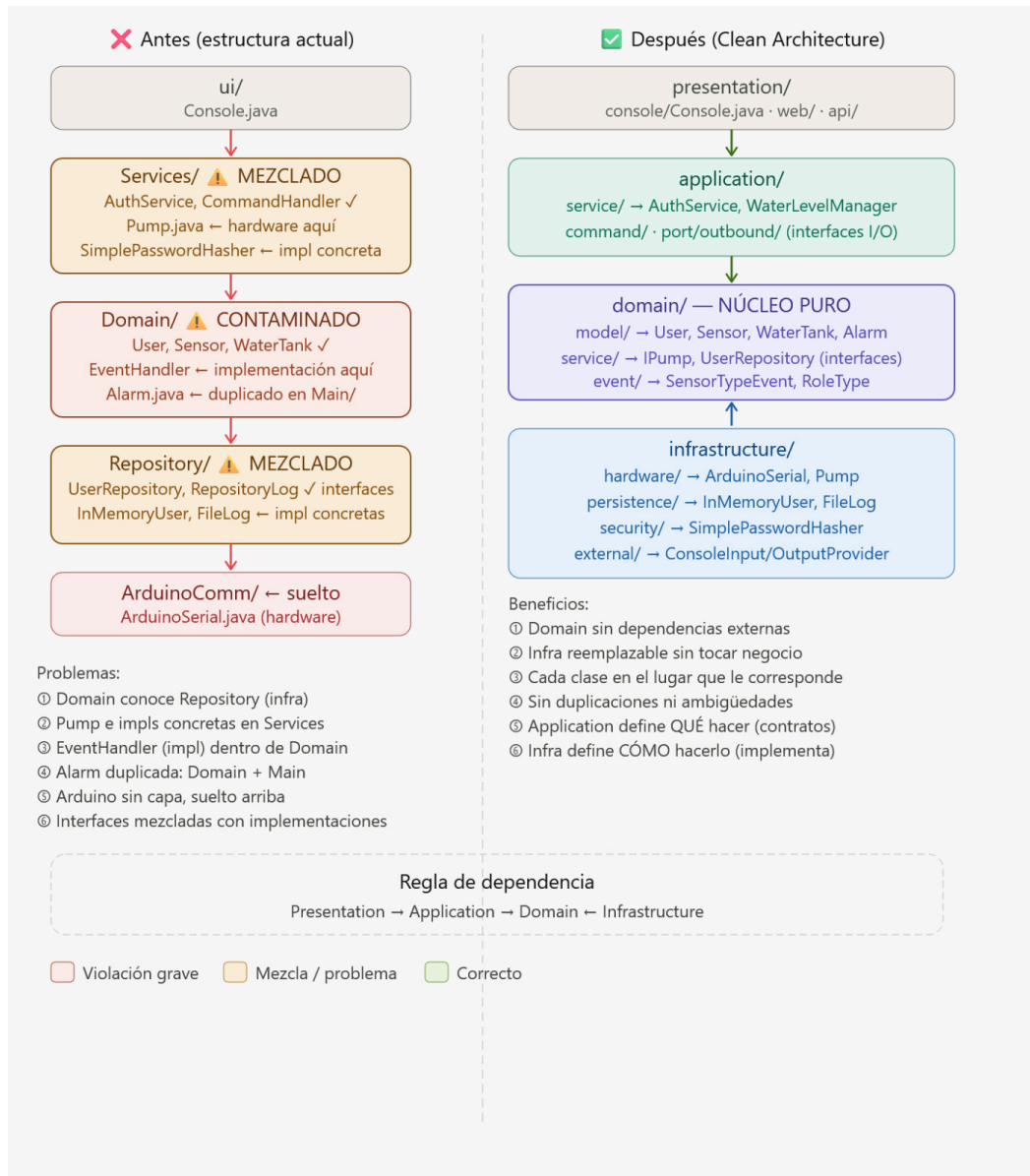


Figura 1. Comparación de la antigua arquitectura implementada con la nueva propuesta.



Figura 2. Presentación de la nueva propuesta de arquitectura implementada cumpliendo con todos lineamientos de Clean Code.

La Regla de Dependencia

Todo el sistema se organiza en círculos concéntricos. La única regla que no se puede violar es: las dependencias del código fuente siempre deben apuntar hacia adentro. Un círculo interior nunca puede saber nada de lo que existe en un círculo exterior — ni importar sus clases, ni usar sus formatos de datos.

Las 4 capas

- 1. Entidades (el núcleo)** — Son los objetos de negocio puro de la empresa. Encapsulan las reglas más críticas y generales. No dependen de nada externo (ni de la base de datos, ni de la UI, ni del framework). Son los que menos deberían cambiar. Ejemplo: una clase Pedido con su lógica de validación de negocio.
- 2. Casos de Uso** — Contienen las reglas de negocio específicas de la aplicación. Orquestan el flujo de datos hacia y desde las Entidades para lograr el objetivo del caso de uso. No saben nada de la UI ni de la base de datos. Ejemplo: AgregarPedidoUseCase, CancelarPedidoUseCase.
- 3. Adaptadores de Interfaz** — Son traductores. Convierten datos del formato conveniente para los Casos de Uso/Entidades al formato conveniente para los sistemas externos (DB, web, etc.). Acá viven los Controllers, Presenters, ViewModels y los Gateways de base de datos. El SQL, por ejemplo, queda encerrado en esta capa.

4. Frameworks y Drivers (la capa más externa) — Son los detalles: el framework web, la base de datos, el servidor, la UI. El libro los llama "detalles" porque son reemplazables. Acá se escribe poco código propio — mayormente es "pegamento" que comunica con la capa interior.

Problemas identificados en la arquitectura pasada

1. El Domain estaba "contaminado"

La carpeta Domain/ tenía dentro a EventHandler.java. El libro dice textualmente que el círculo interno (domain) nunca puede saber nada del exterior. EventHandler es una implementación del patrón Observer, no una regla de negocio. Es decir, es código que dice cómo se distribuyen los eventos, no qué significa un evento para el negocio.

¿Qué puede pasar? Si el día de mañana cambias la forma en que se notifican eventos (por ejemplo, de un Observer sencillo a un sistema de mensajería como RabbitMQ), tenés que tocar el Domain. Y si tocás el Domain, potencialmente rompes todo el núcleo del sistema: las entidades, las reglas de negocio, los contratos. El libro lo dice muy claro: "los cambios en externalidades no deben afectar las entidades".

2. Repository/ mezclaba interfaces con implementaciones

En el mismo paquete vivían juntas UserRepository.java (una interfaz, un contrato) y InMemoryUserRepository.java (la implementación concreta). Esto viola directamente el Principio de Inversión de Dependencia (DIP), que el libro dedica un capítulo entero a explicar.

El libro dice: "las dependencias del código fuente deben apuntar hacia entidades más abstractas". Las interfaces son el contrato que el Domain define; las implementaciones concretas son infraestructura que cumple ese contrato. Si ambas están en el mismo paquete, desde afuera no hay forma de depender solo del contrato sin importar también la implementación. En la práctica, Application terminaba sabiendo que existía InMemoryUserRepository, cuando solo debería saber que existe UserRepository (la interfaz).

3. Services/ tenía código de infraestructura

Pump.java y SimplePasswordHasher.java estaban en el paquete de servicios de aplicación. Pump.java controla hardware físico (un motor, un relé). SimplePasswordHasher es un algoritmo concreto de seguridad. Ninguno de los dos es una regla de negocio: son detalles de cómo se hace algo.

¿Qué puede pasar en la realidad? Supongamos que dejás de usar el Arduino y pasás a controlar la bomba por WiFi o por un PLC diferente. Con la estructura anterior, tenés que entrar al paquete de servicios de aplicación y tocar código que está mezclado con la lógica de casos de uso. Podés romper WaterLevelManager o AuthenticationService sin querer, simplemente porque estaban en el mismo lugar. El libro llama a esto acoplamiento a detalles de implementación, y lo describe como el mayor enemigo de la mantenibilidad.

4. Alarm.java duplicada

Existía en Domain/Alarm.java y también en Main/Alarm.java. Esto es exactamente lo que el libro llama duplicación verdadera: dos clases con el mismo nombre y propósito en dos lugares distintos. El libro distingue entre duplicación accidental (que no hay que eliminar) y verdadera (que hay que resolver sí o sí). Cuando hay dos Alarm.java, no está claro cuál usar, los imports se confunden, y

si corregís un bug en una, la otra queda desactualizada. Con el tiempo el sistema queda en un estado inconsistente sin que nadie lo note.

5. ArduinoComm/ suelto sin capa definida

La carpeta ArduinoComm/ existía al mismo nivel que Domain/, Services/ y Repository/. Desde la perspectiva de Clean Architecture, el hardware es el detalle más externo de todos — debería estar en la capa más exterior. Tenerlo suelto a nivel raíz implica que cualquier parte del código podía importar ArduinoSerial directamente, incluyendo el Domain o los Services, sin ninguna barrera arquitectónica que lo impidiera.

El libro es muy explícito sobre esto: "la web es un detalle, la base de datos es un detalle, mantenemos estas cosas en el exterior donde pueden causar poco daño". Lo mismo aplica al hardware. Si en algún momento alguien importa ArduinoSerial desde un servicio de aplicación, el día que cambies el hardware tenés que rastrear y cambiar código en capas que no deberían saber que Arduino existe.

6. Las dependencias fluían en la dirección incorrecta

El diagrama del documento lo muestra bien: la cadena era ui → Services → Domain → Repository → ArduinoComm. Eso significa que el Domain dependía del Repository. Pero las interfaces de Repository son contratos del dominio: debería ser al revés. El libro lo llama la Regla de Dependencia y es la única regla que no tiene excepciones: "las dependencias del código fuente deben apuntar solo hacia adentro, hacia políticas de nivel superior".

Con las dependencias en la dirección incorrecta, si cambias el sistema de persistencia (por ejemplo, de InMemory a una base de datos real), tenés que tocar el Domain para actualizar imports o referencias. Eso puede romper las reglas de negocio. El libro lo ilustra con el caso de FitNesse: gracias a poner las interfaces en el lugar correcto, pudieron cambiar de almacenamiento en memoria a archivos planos y después a MySQL, sin tocar nada del core de negocio.

Análisis general de la arquitectura de separación de capas

Estructura de capas identificadas:



Figura 3. Representación de la arquitectura por capas junto a sus paquetes y cada separación.

Análisis de cohesión para cada paquete

Paquete	Nivel	Justificación
ArduinoComm	ALTA	Solo responsabilidades de comunicación serial
Domain	ALTA	Solo conceptos del dominio de negocio
Repository	ALTA	Solo acceso a datos y persistencia
Services	MEDIA-ALTA	Múltiples concerns: auth, comandos, managers, contexto
Main	BAJA	Solo bootstrap, no es lógica reutilizable
ui	ALTA	Solo presentación e I/O

Cohesión General: ALTA Los paquetes tienen propósitos claros
Los concerns están separados

Análisis del uso de interfaces

Interfaz	Ubicación	Propósito	¿Bien Usada?
IPump	Domain	Contrato de bomba	SÍ - Pump la implementa, Commands la usan
IWaterSystemStatus	Domain	Lectura de estado	SÍ - WaterLevelManager la implementa, Console la usa
EventListener	Domain	Observer implícito	SÍ - WaterLevelManager la implementa

UserRepository	Repository	CRUD de usuarios	SÍ - bien implementada
RepositoryLog	Repository	Persistencia de logs	SÍ - bien implementada
PasswordHasher	Services	Hash de contraseñas	SÍ - SimplePasswordHasher la implementa
Command	Services	Patrón Command	SÍ - Múltiples comandos la implementan

Evaluación: Las interfaces están bien implementadas

- Abstractan correctamente los contratos
- Permiten polimorfismo
- Facilitan testing

Escalabilidad y extensibilidad de la arquitectura

Puntos Fuertes:

1. Patrón Command bien implementado

Nuevos comandos → Crear class implementando Command
Registrar en CommandHandler

2. Patrón Repository permite cambios de persistencia

Para cambiar a base de datos: nueva implementación de UserRepository

3. Sistema de eventos permite nuevos listeners

Nuevos comportamientos → Crear class implementando EventListener

4. Interfaz IPump

Para simular bomba → Mock

Para controlar motor distinto → Nueva implementación

Puntos Débiles:

1. Comandos hardcodedos en Console

Arrays ADMIN_CMDS y GUEST_CMDS no son dinámicos
Si agrega comando, debe modificar Console

2. **AppContext es un contenedor genérico**

No permite crecer de forma ordenada

Si hay 20 servicios, AppContext crece sin control

3. **WaterLevelManager y TemperatureManager duplican código**

SensorLevelManager es una clase base, no una interfaz

4. **Falta desacoplamiento de I/O**

LoginCommand instancia Scanner

ArduinoSerial instancia Scanner

No hay abstracción de I/O

Conclusión técnica - Calidad de separación de capas

Evaluación general

Aspecto	Calificación	Comentario
Separación de Capas	BUENA	Capas claramente identificadas: UI → Services → Domain → Repository → Hardware
Responsabilidad por Paquete	BUENA	Cada paquete tiene propósito claro
Cohesión	ALTA	No hay mezcla de concerns indebida
Acoplamiento	MODERADO	Console acoplada a AppContext; ArduinoSerial acoplada a WaterLevelSensor
Flujo de Dependencias	CORRECTO	Las dependencias fluyen hacia capas inferiores
Uso de Interfaces	BIEN IMPLEMENTADO	Las interfaces abstractan correctamente
Escalabilidad	MODERADA	Comando pattern es escalable, pero UI debe mejorar
Mantenibilidad	BUENA	Código está organizado, es fácil ubicar responsabilidades

Problemas arquitectónicos detectados

CRÍTICOS:

1. **AppContext como bolsa de servicios (Anti-patrón Service Locator)**

Console hace drilling: `context.getAuth().getCurrentUser()`

Debería haber métodos de fachada en AppContext

2. **Acoplamiento ArduinoSerial → WaterLevelSensor**

ArduinoSerial recibe el sensor específico

¿Qué pasa si queremos múltiples sensores?

IMPORTANTES:

3. **Console toma decisiones de mapeo de comandos**

Arrays hardcodeados no son extensibles

El CommandHandler debería poder listar comandos dinámicamente

4. **Falta abstracción de I/O**

LoginCommand instancia Scanner directamente

ArduinoSerial instancia Scanner

Debería inyectarse como interfaz

MENORES:

5. **WaterLevelManager hereda de SensorLevelManager**

Herencia incompleta: `SensorLevelManager.run()` está vacío

Podría ser una interfaz común

6. **Alarm no implementa EventListener**

Debería reaccionar a eventos de sensor

Actualmente desacoplado del sistema

Recomendaciones inmediatas

Prioridad 1 - Mejorar AppContext:

// Actual:

Java

```
context.getAuth().getCurrentUser().getUserName()
```

// Propuesto:

Java

```
context.getCurrentUserName() // Fachada
```

Prioridad 2 - Inyectar I/O:

Java

```
public interface InputProvider {  
    String readLine();  
}  
  
public LoginCommand(InputProvider input, ...) {  
    this.input = input;  
}
```

Prioridad 3 - Hacer CommandHandler dinámico:

Java

```
List<String> getAvailableCommands() {  
    return new ArrayList<>(commands.keySet());  
}
```

Conclusión final arquitectura

La arquitectura actual es FUNDACIONALMENTE SÓLIDA pero INCOMPLETA en su evolución.

Fortalezas:

1. Separación clara de capas
2. Uso correcto de interfaces
3. Patrón Command implementado adecuadamente
4. Flujo de dependencias correcto

Debilidades:

1. ApplicationContext es un anti-patrón localizador de servicios
2. Algunos acoplamientos de I/O
3. UI no es completamente extensible

Calificación Arquitectónica: 7.5/10

Nivel: Proyecto académico/profesional junior bien estructurado, pero necesita refinamiento para producción empresarial.

A continuación se realizará todas las actividades asignadas para el laboratorio de patrones de diseño donde se evaluará violaciones de los principios SOLID, GRASP y posteriormente identificar los patrones de diseño junto a los problemas que resuelven y la justificación del por qué se implementará cada uno de los nuevos patrones diseñados.

Evaluación SOLID

1. SRP (Single Responsibility Principle) - Responsabilidad Única

Definición: Una clase debe tener una única razón para cambiar, es decir, una sola responsabilidad.

Evaluación General: PARCIALMENTE CUMPLIDO

Ejemplos de Cumplimiento:

AuthenticationService:

Responsabilidad única: Validar credenciales de usuario

Métodos: Solo login() y logout()

Dependencias: Recibe UserRepository y PasswordHasher inyectados

Cohesión: Alta - todos los métodos están relacionados con autenticación

FileLogRepository:

Responsabilidad única: Persistir logs en archivo

Métodos: Solo saveLog()

Cohesión: Alta - solo maneja escritura de logs

Pump:

Responsabilidad única: Controlar el estado de la bomba física

Métodos: turnOn(), turnOff(), getStatus()

Cohesión: Alta - solo gestión de estado de bomba

Ejemplos de Violaciones:

WaterLevelManager:

Múltiples responsabilidades:

- Implementar EventListener (reaccionar a eventos)
- Implementar IWaterSystemStatus (proporcionar estado para UI)
- Heredar de SensorLevelManager (gestión de niveles)
- Contener lógica de automatización de bombeo

Problema: Clase hace demasiadas cosas - viola SRP

Solución propuesta: Separar en WaterLevelEventListener, WaterSystemStatusProvider, PumpAutomationController

Console:

Múltiples responsabilidades:

- Renderizar UI (banners, menús, dashboard)
- Capturar input del usuario
- Resolver comandos (números → strings)
- Controlar flujo de sesión (login/logout)
- Gestionar estado de UI (pausas, colores)

Problema: Clase monolítica de 241 líneas

Solución propuesta: Separar en ConsoleRenderer, InputHandler, SessionController

Main:

Responsabilidad: Bootstrap de la aplicación

Problema: Contiene lógica de configuración compleja (60 líneas)

Solución: Usar framework de DI o patrón Builder

2. OCP (Open/Closed Principle) - Abierto/Cerrado

Definición: Las entidades de software deben estar abiertas para extensión pero cerradas para modificación.

Evaluación General: MODERADAMENTE CUMPLIDO

Ejemplos de Cumplimiento:

Command Pattern:

Abierto para extensión: Nuevos comandos implementan Command sin modificar CommandHandler

Cerrado para modificación: CommandHandler.execute() no cambia al agregar comandos

Ejemplo: Agregar TemperatureControlCommand requiere solo nueva clase

Repository Pattern:

Abierto para extensión: Nuevas implementaciones de UserRepository (ej. DatabaseUserRepository)

Cerrado para modificación: AuthenticationService no cambia

Ejemplo: Cambiar de memoria a base de datos requiere solo nueva implementación

Ejemplos de Violaciones:

Console con arrays hardcodeados:

Java

```
private static final String[] ADMIN_CMDS = {  
    "ver_nivel", "prender_bomba", "apagar_bomba", "logs", "logout", "exit"  
};
```

Problema: Para agregar comando, se debe modificar Console

Violación: No está cerrado para modificación

Solución: Hacer que CommandHandler proporcione lista dinámica de comandos

AuthorizationService:

Java

```
public boolean hasPermission(User user, String code) {  
    if (user.getRole() == RoleType.ADMIN) {  
        return true;  
    }  
    return false;  
}
```

Problema: Lógica hardcodeada para roles

Violación: No extensible para nuevos permisos específicos

Solución: Patrón Strategy o mapa de permisos

3. LSP (Liskov Substitution Principle) - Sustitución de Liskov

Definición: Los objetos de una subclase deben poder sustituir a objetos de la clase base sin alterar el comportamiento esperado.

Evaluación General: CUMPLIDO

Ejemplos de Cumplimiento:

Sensor Hierarchy:

WaterLevelSensor y TemperatureSensor extienden Sensor
Ambos pueden ser tratados como Sensor sin problemas
handleSensor() se comporta correctamente en ambas subclases

Repository Implementations:

InMemoryUserRepository implementa UserRepository
FileLogRepository implementa RepositoryLog
Pueden sustituirse sin cambiar comportamiento esperado

Command Implementations:

Todos los comandos implementan Command
CommandHandler trata todos igual: command.execute(context, arg)

Ejemplos de Violaciones:

WaterLevelManager hereda de SensorLevelManager:

SensorLevelManager.run() está vacío (no implementado)
WaterLevelManager no usa realmente la herencia
Problema: No cumple contrato de la clase base
Solución: Cambiar a composición o interfaz común

4. ISP (Interface Segregation Principle) - Segregación de Interfaces

Definición: Los clientes no deben depender de interfaces que no usan.

Evaluación General: BIEN CUMPLIDO

Ejemplos de Cumplimiento:

IPump:

Métodos: turnOn(), turnOff(), getStatus()
Clientes: PumpOnCommand, PumpOffCommand, WaterLevelManager
Uso: Todos los clientes usan todos los métodos
Evaluación: Interface apropiada y cohesiva

IWaterSystemStatus:

Métodos: getCurrentDistance(), getCurrentPercentage(), isPumpActive(), hasData()

Cliente: Console (UI)

Uso: UI necesita leer estado completo del sistema

Evaluación: Interface bien segregada para capa de presentación

Command:

Método: execute(AppContext context, String arg)

Clientes: CommandHandler y todas las implementaciones

Uso: Contrato mínimo y específico

Evaluación: Perfecto ejemplo de ISP

Ejemplos de Violaciones:

AppContext expone múltiples servicios:

Java

```
public Authenticator getAuth()    { return auth; }  
public EventHandler getHandler() { return handler; }  
public RepositoryLog getLogger() { return logger; }
```

Problema: Console solo usa getAuth(), pero depende de toda la interfaz

Violación: Depende de servicios que no usa

Solución: Interfaces específicas o patrón Fachada

5. DIP (Dependency Inversion Principle) - Inversión de Dependencias

Definición: Los módulos de alto nivel no deben depender de módulos de bajo nivel. Ambos deben depender de abstracciones.

Evaluación General: PARCIALMENTE CUMPLIDO

Ejemplos de Cumplimiento:

DIP en IPump:

Módulo alto: WaterLevelManager (lógica de negocio)

Módulo bajo: Pump (implementación concreta)

Abstracción: IPump (interface en domain)

Dependencia: WaterLevelManager depende de IPump, no de Pump

Evaluación: DIP cumplido correctamente

DIP en Repositories:

Módulo alto: AuthenticationService

Módulo bajo: InMemoryUserRepository

Abstracción: UserRepository

Dependencia: AuthenticationService recibe UserRepository inyectado

Evaluación: DIP cumplido

DIP en PasswordHasher:

Módulo alto: AuthenticationService

Módulo bajo: SimplePasswordHasher

Abstracción: PasswordHasher

Dependencia: Interface inyectada correctamente

Ejemplos de Violaciones:

ArduinoSerial depende de WaterLevelSensor concreto:

Java

```
public ArduinoSerial(String nombrePuerto, WaterLevelSensor sensor) {  
    this.sensorDelDominio = sensor;  
}
```

Problema: Acoplado a tipo específico de sensor

Violación: Módulo de infraestructura depende de entidad de dominio concreta

Solución: Depender de interfaz Sensor o callback

InMemoryUserRepository instancia SimplePasswordHasher:

Java

```
SimplePasswordHasher hasher = new SimplePasswordHasher();
```

Problema: Repository (bajo nivel) instancia implementación concreta

Violación: No usa inyección de dependencias

Solución: Recibir PasswordHasher en constructor

Console instancia Scanner:

Java

```
private final Scanner scanner = new Scanner(System.in);
```

Problema: UI acoplada a System.in

Violación: No permite testing o diferentes fuentes de input

Solución: Inyectar InputProvider interface

Evaluación GRASP

1. Experto en Información (Information Expert)

Definición: Asignar responsabilidad al objeto que tiene la información necesaria.

Evaluación General: BIEN APLICADO

Ejemplos de Cumplimiento:

WaterTank.calculatePercentage():

Información: Conoce totalHeight

Responsabilidad: Calcular porcentaje basado en distancia

Justificación: Tiene la información necesaria (altura total)

User.isActive():

Información: Conoce isActive

Responsabilidad: Determinar si usuario está activo

Justificación: Encapsula su propio estado

Pump.getStatus():

Información: Conoce isActive

Responsabilidad: Reportar estado de la bomba

Justificación: Tiene el estado interno

Ejemplos de Violaciones:

Console conoce estructura de User:

Java

```
String role = context.getAuth().getCurrentUser().getRole().toString();
```

Problema: Console (UI) conoce detalles de User

Violación: UI no debería conocer estructura de dominio

Solución: User debería tener método getRoleName()

2. Creador (Creator)

Definición: Asignar responsabilidad de creación a la clase que usa el objeto creado.

Evaluación General: MODERADAMENTE APLICADO

Ejemplos de Cumplimiento:

Main crea todos los objetos:

Usa: Main orquesta toda la aplicación

Crea: Todos los servicios, repositorios, sensores

Justificación: Main es el punto de entrada que usa todo

InMemoryUserRepository crea User y Credential:

Usa: Repository gestiona usuarios

Crea: Instancias de User y Credential

Justificación: Repository conoce la estructura de datos

Ejemplos de Violaciones:

ArduinoSerial no crea WaterLevelSensor:

Problema: Recibe sensor como parámetro

Violación: Otro objeto (Main) crea y pasa el sensor

Solución: Podría crear sensor interno si es su responsabilidad

3. Bajo Acoplamiento (Low Coupling)

Definición: Minimizar dependencias entre clases.

Evaluación General: MODERADO

Ejemplos de Cumplimiento:

Command Pattern:

Acoplamiento: CommandHandler solo conoce Command interface

Beneficio: Nuevos comandos no afectan CommandHandler

Evaluación: Bajo acoplamiento logrado

Repository Pattern:

Acoplamiento: domain.service dependen de interfaces, no implementaciones

Beneficio: Cambiar persistencia no afecta lógica de negocio

Evaluación: Bajo acoplamiento logrado

Ejemplos de Violaciones:

Console → ApplicationContext drilling:

Acoplamiento: Console conoce estructura interna de ApplicationContext

Problema: context.getAuth().getCurrentUser().getRole()

Solución: ApplicationContext debería tener método isUserAdmin()

WaterLevelManager hereda de SensorLevelManager:

Acoplamiento: Herencia crea acoplamiento fuerte

Problema: Cambios en SensorLevelManager afectan WaterLevelManager

Solución: Composición sobre herencia

4. Alta Cohesión (High Cohesion)

Definición: Las responsabilidades de una clase deben estar relacionadas.

Evaluación General: MODERADA

Ejemplos de Cumplimiento:

AuthenticationService:

Responsabilidades: Login, logout, verificación de credenciales

Relación: Todas relacionadas con autenticación de usuarios

Cohesión: Alta

Pump:

Responsabilidades: Encender, apagar, reportar estado

Relación: Todas relacionadas con control de bomba

Cohesión: Alta

Ejemplos de Violaciones:

WaterLevelManager:

Responsabilidades mezcladas: Event listening, status reporting, automation logic

Relación: Débil - hace demasiadas cosas diferentes

Cohesión: Baja

Console:

Responsabilidades mezcladas: Rendering, input, session management

Relación: Todas relacionadas con UI, pero podrían separarse

Cohesión: Media

5. Controlador (Controller)

Definición: Manejar eventos del sistema y delegar a objetos apropiados.

Evaluación General: BIEN APLICADO

Ejemplos de Cumplimiento:

CommandHandler:

Rol: Recibe comandos del usuario

Delegación: Encuentra y ejecuta el Command apropiado

Justificación: Controlador puro del patrón Command

Authenticator:

Rol: Coordina autenticación y autorización

Delegación: Usa AuthenticationService y AuthorizationService

Justificación: Controlador de seguridad

Ejemplos de Violaciones:

Console actúa como controlador:

Problema: Console recibe input y decide qué hacer

Violación: UI no debería ser controlador

Solución: Console debería delegar a CommandHandler

6. Polimorfismo (Polymorphism)

Definición: Usar polimorfismo para manejar variaciones.

Evaluación General: BIEN APLICADO

Ejemplos de Cumplimiento:

Command implementations:

Polimorfismo: CommandHandler trata todos los comandos igual

Beneficio: Nuevo comando sin cambiar CommandHandler

Evaluación: Polimorfismo bien usado

Repository implementations:

Polimorfismo: AuthenticationService usa cualquier UserRepository

Beneficio: Cambiar implementación sin afectar domain.service

Evaluación: Polimorfismo bien usado

7. Fabricación Pura (Pure Fabrication)

Definición: Crear clases que no representan conceptos del dominio pero mejoran cohesión y acoplamiento.

Evaluación General: LIMITADAMENTE APLICADO

Ejemplos de Cumplimiento:

AppContext:

Rol: Contenedor de servicios (no es concepto de dominio)

Beneficio: Centraliza dependencias

Evaluación: Fabricación pura para DI manual

CommandHandler:

Rol: Dispatcher de comandos (no es concepto de dominio)

Beneficio: Separa lógica de routing

Evaluación: Fabricación pura bien aplicada

Ejemplos de Violaciones:

Falta fabricación pura para logging:

Problema: Cada clase que loguea instancia RepositoryLog

Solución: Servicio de logging centralizado

8. Indirección (Indirection)

Definición: Usar intermediario para reducir acoplamiento.

Evaluación General: MODERADAMENTE APLICADO

Ejemplos de Cumplimiento:

EventHandler:

Rol: Intermediario entre publishers y subscribers

Beneficio: Desacopla sensores de listeners

Evaluación: Indirección bien aplicada

CommandHandler:

Rol: Intermediario entre UI y lógica de comandos

Beneficio: UI no conoce implementaciones concretas

Evaluación: Indirección bien aplicada

9. Variaciones Protegidas (Protected Variations)

Definición: Proteger elementos de variaciones en otros elementos.

Evaluación General: MODERADO

Ejemplos de Cumplimiento:

Interfaces protegen de cambios:

Protección: IPump protege services de cambios en Pump

Beneficio: Cambiar implementación de bomba no afecta lógica

Evaluación: Variaciones protegidas bien aplicadas

Repository interfaces:

Protección: UserRepository protege de cambios de persistencia

Beneficio: Cambiar de memoria a BD no afecta AuthenticationService

Evaluación: Variaciones protegidas bien aplicadas

Ejemplos de Violaciones:

Console conoce comandos específicos:

Problema: Arrays hardcoded no protegen de nuevos comandos

Solución: CommandHandler debería exponer comandos disponibles

Conclusión SOLID y GRASP

Una vez analizados todos los principios GRASP y SOLID llegamos a la siguiente conclusión:

Principio	Cumplimiento	Puntuación
SRP	Parcial	6/10
OCP	Moderado	7/10
LSP	Bueno	8/10
ISP	Bueno	8/10
DIP	Parcial	6/10
Experto en Información	Bueno	8/10
Creador	Moderado	7/10
Bajo Acoplamiento	Moderado	6/10
Alta Cohesión	Moderado	6/10

Controlador	Bueno	8/10
Polimorfismo	Bueno	8/10
Fabricación Pura	Limitado	5/10
Indirección	Moderado	7/10
Variaciones Protegidas	Moderado	7/10

Puntuación Global: 6.9/10

Análisis de Fortalezas

SOLID:

LSP e ISP bien aplicados - interfaces limpias y herencia correcta
DIP parcialmente logrado con interfaces de repositorios y pump

GRASP:

Polimorfismo excelente en Command y Repository patterns
Controlador bien aplicado en CommandHandler y Authenticator
Indirección logrado con EventHandler

Problemas Críticos Detectados

SRP Violations: WaterLevelManager y Console tienen múltiples responsabilidades
DIP Violations: Acoplamientos concretos en ArduinoSerial y InMemoryUserRepository
Low Coupling Issues: Console hace drilling en ApplicationContext
High Cohesion Problems: Clases monolíticas con responsabilidades mezcladas

Recomendaciones Prioritarias

Inmediatas (1-2 semanas):

Refactorizar WaterLevelManager: Separar en clases específicas por responsabilidad
Implementar DIP completo: Inyectar dependencias en InMemoryUserRepository
Crear Fachada en ApplicationContext: Métodos específicos en lugar de getters genéricos

Mediano Plazo (1-2 meses):

Separar Console: Dividir en UIRenderer, InputController, SessionManager
Implementar Command Discovery: Hacer CommandHandler dinámico
Abstraer I/O: Interfaces para input/output

Largo Plazo:

Framework de DI: Reemplazar inyección manual por Spring o similar

Event Bus: Mejorar sistema de eventos con bus desacoplado

Testing Framework: Implementar tests unitarios para validar principios

Beneficios Esperados:

Escalabilidad: Sistema adaptable a nuevos requerimientos

Mantenibilidad: Código más modular y testable

Extensibilidad: Nuevas funcionalidades sin modificar código existente

Patrones de diseño

Se tuvo en cuenta para este avance los patrones de diseño clásicos o llamados también “Gang of Four” (GoF) que se dividen en tres categorías principales según el propósito y el funcionamiento de cada uno:

Patrones Creacionales:

Se enfocan en los mecanismos de creación de objetos, intentando crear objetos de una manera adecuada a la situación para evitar dependencias innecesarias.

- Singleton: Asegura que una clase tenga una única instancia en todo el programa.
- Factory Method: Proporciona una interfaz para crear objetos en una superclase, pero permite a las subclasses alterar el tipo de objetos que se crearán.
- Builder: Permite construir objetos complejos paso a paso.

Patrones Estructurales:

Explican cómo ensamblar objetos y clases en estructuras más grandes, manteniendo estas estructuras flexibles y eficientes.

- Adapter: Permite que objetos con interfaces incompatibles trabajen juntos (como un adaptador de enchufe físico).
- Decorator: Permite añadir funcionalidades a objetos individuales de forma dinámica sin alterar su estructura.
- Facade: Proporciona una interfaz simplificada a una biblioteca, un framework o cualquier otro grupo complejo de clases.

Patrones de Comportamiento:

Se encargan de la comunicación entre objetos, definiendo cómo interactúan y se reparten las responsabilidades.

- Observer: Permite que un objeto notifique a otros objetos sobre cambios en su estado.

- Strategy: Define una familia de algoritmos, los encapsula y los hace intercambiables en tiempo de ejecución.
- State: Permite que un objeto altere su comportamiento cuando su estado interno cambia, evitando largos condicionales `if-else`.

De los cuáles se identificaron los siguientes en el proyecto:

Patrones Creacionales:

Factory Method: NO PRESENTE

No hay jerarquías donde subclases decidan qué instancias crear
Main instancia directamente todos los objetos sin delegación

Abstract Factory: NO PRESENTE

No hay familias de objetos relacionados que necesiten creación coordinada

Builder: NO PRESENTE

La construcción de objetos es simple, no requiere builders complejos

Prototype: NO PRESENTE

No hay clonación de objetos complejos

Singleton: INCIPIENTE

FileLogRepository podría considerarse singleton por su estado estático interno
Sin embargo, se instancia normalmente en Main

Patrones Estructurales:

Adapter: NO PRESENTE

No hay adaptación de interfaces incompatibles

Bridge: NO PRESENTE

No hay separación de abstracción e implementación

Composite: NO PRESENTE

No hay estructuras jerárquicas de objetos

Decorator: NO PRESENTE

No hay extensión dinámica de funcionalidades

Facade: INCIPIENTE

AppContext actúa como fachada para acceder a servicios (getAuth(), getHandler())
Sin embargo, expone getters directos sin encapsulación real

Flyweight: NO PRESENTE

No hay objetos compartidos para eficiencia de memoria

Proxy: NO PRESENTE

No hay intermediarios para controlar acceso

Patrones de Comportamiento:

Chain of Responsibility: NO PRESENTE

No hay cadenas de manejadores para requests

Command: PRESENTE

Command interface, CommandHandler, PumpOnCommand, LoginCommand
Patrón completamente implementado para encapsular requests

Iterator: NO PRESENTE

No hay traversal de colecciones complejas

Mediator: INCIPIENTE

EventHandler actúa como mediador entre sensores y listeners
Sin embargo, es más un Observer que un Mediator puro

Memento: NO PRESENTE

No hay captura y restauración de estado interno

Observer: PARCIALMENTE PRESENTE

EventHandler, EventListener, Event
Sensores notifican cambios, pero implementación limitada

State: NO PRESENTE

No hay cambios de comportamiento basados en estado interno

Strategy: NO PRESENTE

No hay algoritmos intercambiables encapsulados

Template Method: INCIPIENTE

Sensor.handleSensor() define template, pero subclasses lo implementan completamente

Visitor: NO PRESENTE

No hay operaciones sobre estructuras de objetos

Resumen de la identificación GoF: De 23 patrones, solo Command está completamente implementado. Observer está parcialmente presente, y hay indicios incipientes de Facade, Mediator y Template Method.

Evaluación de patrones de diseño críticos

Patrón Repository

Implementación Actual:

UserRepository: Interface con métodos save(), findByUserName(), findById()

InMemoryUserRepository: Implementación en memoria con HashMap<UUID, User>

FileLogRepository: Implementación para logs con escritura a archivo

Evaluación de Desacoplamiento:

Aislamiento Correcto:

AuthenticationService depende solo de UserRepository interface

Lógica de dominio (AuthenticationService) no conoce detalles de persistencia

Cambiar de InMemoryUserRepository a DatabaseUserRepository no requiere cambios en services

Fugas de Implementación Detectadas:

1. InMemoryUserRepository instancia SimplePasswordHasher:

Java

```
SimplePasswordHasher hasher = new SimplePasswordHasher();
```

Problema: Repository conoce implementación concreta de hashing

Violación: DIP - debería recibir PasswordHasher inyectado

Impacto: Repository acoplado a algoritmo de hash específico

2. FileLogRepository tiene estado estático hardcodeado:

Java

```
private static final String LOG_FILE = "../historial_tanque.txt";
```

Problema: Ruta hardcodeda, no configurable

Violación: OCP - cerrado para modificación de ubicación de logs

Conclusión Repository: 7/10 - Patrón bien aplicado pero con fugas de DIP y configuración.

Patrón Command

Implementación Actual:

Command: Interface con `execute(AppContext context, String arg)`

CommandHandler: Registry y dispatcher de comandos

Implementaciones: PumpOnCommand, PumpOffCommand, LoginCommand, ViewLevelCommand, ViewLogsCommand

Evaluación de Diseño:

Aspectos Positivos:

Encapsulación correcta de acciones en objetos

CommandHandler desacoplado de implementaciones concretas

Fácil extensión: nuevos comandos sin modificar dispatcher

AppContext como receptor común

Errores de Diseño Críticos:

1. Acoplamiento excesivo a AppContext:

Java

```
public void execute(AppContext context, String arg)
```

Problema: Todos los comandos reciben AppContext completo

Violación: ISP - comandos simples no necesitan todos los servicios

Impacto: ViewLogsCommand no usa `context.getAuth()`, pero lo recibe

2. CommandHandler no valida permisos:

Problema: Dispatcher ejecuta cualquier comando sin verificar autorización

Violación: Seguridad - cualquier usuario puede ejecutar PumpOnCommand

Solución: CommandHandler debería verificar permisos antes de ejecutar

3. LoginCommand instancia Scanner directamente:

Java

```
Scanner scanner = new Scanner(System.in);
```

Problema: Comando acoplado a entrada específica

Violación: DIP - no permite testing o diferentes inputs

Impacto: Comando no testable unitariamente

4. Comandos mezclan lógica de UI y negocio:

LoginCommand maneja input/output directamente

Problema: Comando debería solo ejecutar lógica, no interactuar con usuario

Violación: SRP - comandos hacen demasiado

Conclusión Command: 6/10 - Patrón implementado pero con acoplamientos problemáticos y falta de seguridad.

Patrón Strategy & Observer

Evaluación patrón Strategy:

NO IMPLEMENTADO

Lógica Actual: WaterLevelManager tiene lógica hardcodeada de bombeo:

Java

```
if (porcentaje >= 90.0f && pump.getStatus()) {  
    pump.turnOff();  
} else if (porcentaje <= 10.0f && !pump.getStatus()) {  
    pump.turnOn();  
}
```

Problema: Algoritmo fijo, no extensible

Falta: Estrategias intercambiables como AggressivePumpStrategy, EcoPumpStrategy, TemperatureAwarePumpStrategy

Evaluación: 0/10 - Patrón no aplicado

Evaluación patrón Observer:

PARCIALMENTE IMPLEMENTADO

Componentes Presentes: EventHandler, EventListener, Event

Funcionamiento: Sensores emiten eventos, WaterLevelManager escucha

Aspectos Positivos:

Desacoplamiento básico entre sensores y listeners

EventHandler maneja suscripciones correctamente

Extensible: nuevos listeners sin modificar publishers

Limitaciones Críticas:

1. Observer limitado a sensores:

Solo WaterLevelManager implementa EventListener

Problema: Alarmas, UI, otros componentes no reaccionan a eventos

Falta: Sistema de notificaciones completo

2. Eventos no tipados fuertemente:

Java

```
public void onEvent(Event event)
```

Problema: Listener debe hacer instanceof o switch en event.getType()

Violación: LSP - diferentes tipos de eventos no son sustituibles

3. Falta de Event Bus:

Problema: EventHandler es singleton implícito compartido

Solución: Event Bus centralizado para comunicación global

Conclusión Strategy & Observer: Strategy 0/10, Observer 5/10 - Observer incipiente, Strategy ausente.

Propuesta implementación nuevos patrones y justificación

Patrón 1: Strategy - Estrategias de Bombeo

Problema Actual: WaterLevelManager tiene lógica de bombeo hardcodeada con umbrales fijos (10% encender, 90% apagar). No permite diferentes estrategias de control (agresiva, conservadora, temperatura-aware).

Patrón Propuesto: Strategy

Interface PumpStrategy con método shouldActivatePump(WaterLevelData data)

Implementaciones: ConservativePumpStrategy, AggressivePumpStrategy, TemperatureAwarePumpStrategy

WaterLevelManager recibe estrategia inyectada

Beneficio SOLID/GRASP:

OCP: Nuevas estrategias sin modificar WaterLevelManager

DIP: WaterLevelManager depende de abstracción PumpStrategy

Polimorfismo: Estrategias intercambiables en runtime

Alta Cohesión: Lógica de decisión separada de ejecución

Patrón 2: Observer Completo - Sistema de Notificaciones

Problema Actual: Solo WaterLevelManager reacciona a eventos de sensores. Alarm no está integrada, UI no recibe notificaciones automáticas, falta logging automático de eventos críticos.

Patrón Propuesto: Observer con Event Bus

EventBus central como publisher/subscriber avanzado

Alarm implementa EventListener para reaccionar a eventos de nivel crítico

UI se suscribe para actualizaciones automáticas

LoggerListener como listener automático para auditoría

Beneficio SOLID/GRASP:

Bajo Acoplamiento: Componentes no conocen unos a otros, solo al bus

Alta Cohesión: Cada componente enfocado en su responsabilidad

Indirección: EventBus como intermediario desacoplado

Variaciones Protegidas: Nuevos listeners sin modificar publishers

Patrón 3: Factory Method - Creación de Sensores

Problema Actual: Main instancia sensores directamente (new WaterLevelSensor(handler)). No hay abstracción para crear diferentes tipos de sensores o configuraciones.

Patrón Propuesto: Factory Method

Interface SensorFactory con método createSensor(EventHandler handler)

Implementaciones: BasicSensorFactory, AdvancedSensorFactory

Main usa factory para crear sensores apropiados

Beneficio SOLID/GRASP:

Experto en Información: Factory conoce detalles de creación

Bajo Acoplamiento: Main no conoce tipos concretos de sensores

OCP: Nuevos tipos de sensores sin modificar código cliente

Fabricación Pura: Factory no es concepto de dominio pero mejora diseño

Caso de uso implementado

Se integró el sensor ultrasónico **HC-SR04** para la medición del nivel y el módulo **DS18B20** para el monitoreo térmico, garantizamos una gestión eficiente del agua. La optimización del cableado mediante el uso estratégico de los **rieles de la protoboard** no solo mejora la estética visual, sino que facilita la adición de futuros componentes. El uso del relé para controlar la **bomba sumergible**, alimentada por la fuente externa, asegura que la electrónica sensible permanezca protegida de las cargas de alta potencia. Finalmente, la implementación de una **lógica de control**

modular permite que el sistema crezca sin fricciones, convirtiendo una idea en una solución tecnológica de nivel profesional.